

SOFTWARE REQUIREMENTS SPECIFICATION

for

SwiftX — FinTech Financial Inclusion Platform

Zero-Fee Remittance, Digital Wallets & AI-Powered Financial Guidance

Version 1.0

Prepared by

SwiftX Development Team

May 2026

Prepared in accordance with IEEE Std 830-1998

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Intended Audience and Reading Suggestions.....	4
1.3 Project Scope.....	4
1.4 References.....	4
1.5 SRS Structure Overview.....	4
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Features.....	5
2.3 User / Stakeholder Classes and Characteristics.....	5
2.4 Operating Environment and System Architecture.....	6
2.4.1 PWA Frontend (Next.js 14).....	6
2.4.2 Backend and Database (Supabase / PostgreSQL).....	6
2.4.3 Authentication (Supabase Auth).....	6
2.4.4 FX Rate Engine.....	6
2.4.5 Blockchain Ledger and Polygon Anchoring.....	6
2.4.6 AI Insights Engine (Anthropic Claude API).....	6
2.5 Design and Implementation Constraints.....	7
2.6 Business Requirements.....	7
2.6.1 Business Opportunity.....	7
2.6.2 Business Objectives and Success Criteria.....	7
2.6.3 Customer or Market Needs.....	7
2.6.4 Business Risks.....	7
2.7 User Documentation.....	8
2.8 Assumptions and Dependencies.....	8
3. System Features.....	9
3.1 Zero-Fee Remittance.....	9
3.1.1 FX Breakdown Display.....	9
3.1.2 Hash-Chained Transaction Receipt.....	9
3.1.3 Multi-Currency Wallet.....	9
3.2 User Authentication and KYC Verification.....	10
3.2.1 Authentication Requirements.....	10
3.2.2 KYC Verification Requirements.....	10
3.3 AI-Powered Financial Insights.....	11
3.3.1 AI Insights Functional Requirements.....	11
3.4 AML Compliance and Admin Dashboard.....	11
3.4.1 AML Rules Engine Requirements.....	11
3.5 Micro-Investment Products.....	12
3.5.1 Micro-Investment Functional Requirements.....	12
3.6 Multilingual Interface and Voice Input.....	13
3.6.1 Multilingual and Voice Functional Requirements.....	13
4. Non-Functional Requirements.....	14
4.1 Process Requirements.....	14
4.1.1 Management Requirements.....	14
4.1.2 Implementation Requirements.....	14
4.1.3 Standards Requirements.....	14
4.2 Product Requirements.....	15
4.2.1 Usability Requirements.....	15

4.2.2 Performance Requirements.....	15
4.2.3 Reliability Requirements.....	15
4.2.4 Availability.....	15
4.2.5 Platform Constraints.....	15
4.2.6 Modifiability.....	15
4.3 External Requirements.....	16
4.3.1 Business Rules.....	16
4.3.2 Legal Constraints.....	16
4.3.3 Economic Constraints.....	16
4.3.4 Interoperability Requirements.....	16
4.3.4.1 User Interfaces.....	16
5. Validation Criteria.....	17
Appendix A: Architecture Diagrams.....	18
Appendix B: Glossary and Acronyms.....	19
Appendix C: Word Index.....	20
Appendix D: Traceability Matrix.....	21
Appendix E: Authorship.....	22

1. Introduction

1.1 Purpose

SwiftX version 1.0 is a mobile-first financial inclusion platform designed to provide zero-fee remittance services, secure multi-currency digital wallets, and AI-powered financial guidance for migrant workers and underserved populations. The platform is accessible via a Progressive Web Application (PWA) with wireless Internet connectivity, enabling users to send money across borders, manage balances in multiple currencies, and receive personalized financial insights from any mobile device. As SwiftX evolves in its software product line, it is expected to expand to include additional financial services such as micro-investment products, savings automation, insurance bundles, and credit scoring for the unbanked.

1.2 Intended Audience and Reading Suggestions

This document is primarily written for the developers, project managers, compliance officers, and testers of the SwiftX platform, as it focuses on the required functionality, analysis, and design of the system. It is recommended that the SRS Structure Overview section be read first before proceeding to the sections most pertinent to each reader type. Marketing staff will be briefed on product features by the development team. A user manual and in-app onboarding tutorial will be provided for end users, with special attention to low-literacy and multilingual users.

1.3 Project Scope

The first generation of the SwiftX software product line will focus primarily on cross-border remittance and digital wallet features, which are immediately understandable and valuable to the target market of migrant workers sending money home. The core demo persona is Ravi, a construction worker in Dubai (holding USD), who sends money to his mother in Kerala, India (receiving INR). As users become comfortable with SwiftX, new features will be added to expand financial inclusion including micro-investment products, savings tools, and credit building making SwiftX a comprehensive financial companion for the underbanked.

1.4 References

IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)
"Software Engineering: A Practitioner's Approach (SEPA)" by R. S. Pressman, McGraw-Hill, 6th Edition.
Reserve Bank of India (RBI) Money Transfer Service Scheme (MTSS) Guidelines, 2023.
Financial Action Task Force (FATF) Recommendations for AML/KYC, 2024.
General Data Protection Regulation (GDPR), European Union, 2018.
Personal Data Protection Bill (PDPB), India, 2023.

1.5 SRS Structure Overview

Prior to the introduction, the table of contents shows how the SRS is organized. A revision history is included. The introduction states SwiftX's purpose, the SRS intended audience, the project scope, and useful references. The overall description section covers product perspective, product features, user classes, system components, constraints, business requirements, and assumptions. Section 3 categorizes system features; each feature section includes use cases followed by specific sub-features and functional requirements. Section 4 groups all non-functional requirements. The remaining sections address validation criteria. The appendix includes diagrams, a glossary, index, and traceability information.

2. Overall Description

2.1 Product Perspective

SwiftX version 1.0 is a new-generation financial inclusion platform built on a mobile-first Progressive Web Application architecture. The system is conceptualized to address the \$700 billion global remittance market, where an estimated 250 million migrant workers pay an average of 6.2% in fees per transfer. SwiftX eliminates per-transaction fees by using a multi-currency liquidity pool model: money moves wallet-to-wallet inside the platform's ledger rather than crossing bank rails for each individual transaction.

The platform's architecture consists of a Next.js PWA frontend, a Supabase (PostgreSQL) backend with Row Level Security, a SHA-256 hash-chained transaction ledger with optional Polygon blockchain anchoring for public verifiability, an Anthropic Claude AI integration for personalized financial insights, and a live FX rate engine. Communication between client and server is conducted over HTTPS with encrypted tokens. The system is hosted on AWS for global availability and scalability.

Figure 2.1.1 (see Appendix A) illustrates the SwiftX high-level deployment architecture.

2.2 Product Features

The first generation of the SwiftX software product line will focus primarily on cross-border remittance and digital wallet features. Key features include:

- Zero-fee remittance: Wallet-to-wallet transfers with no per-transaction fee, showing a transparent FX breakdown screen with live exchange rates and disclosed margin.
- Multi-currency digital wallets: Support for USD, AED, INR, and additional currencies, with real-time balance updates using Supabase realtime subscriptions.
- Cryptographic transaction ledger: SHA-256 hash-chained immutable ledger with periodic anchoring to Polygon Amoy public blockchain for external verifiability.
- AI-powered financial insights: Personalized guidance, savings recommendations, and investment suggestions delivered in the user's language via Claude AI.
- KYC / AML compliance: Webcam-based or document-upload KYC verification, automated AML rules engine with velocity checks and threshold alerts.
- Micro-investment products: Systematic Investment Plan (SIP) initiation for small recurring investments accessible to the underbanked.
- Multilingual interface: English and Hindi support (extensible to additional languages) with voice command input via Web Speech API.
- Admin compliance dashboard: Real-time liquidity pool monitoring, AML flag management, and full audit log viewer for compliance officers.

2.3 User / Stakeholder Classes and Characteristics

- a. Sender (Migrant Worker):** The primary end user, such as Ravi — a construction worker in Dubai sending money home to India. Likely to be on a mobile device with limited data, possibly low financial literacy, preferring simple interfaces in their native language.
- b. Recipient (Family Member in Home Country):** Receives INR to their wallet and withdraws via UPI/IMPS. May have limited smartphone experience and require a simple, localized interface.
- c. Compliance Officer / Admin:** Monitors AML flags, reviews transaction audit logs, manages liquidity pool balances, and ensures adherence to RBI MTSS guidelines and FATF recommendations.
- d. Business Stakeholders / Investors:** Evaluate platform viability, revenue model sustainability (FX spread + float interest + premium features), and regulatory compliance posture.

2.4 Operating Environment and System Architecture

SwiftX is a cloud-native application accessible on any modern mobile browser without installation. The system communicates over HTTPS and uses Supabase for authentication, database, and real-time subscriptions. The following subsections describe the major system components.

2.4.1 PWA Frontend (Next.js 14)

The user-facing application is a Progressive Web Application built with Next.js 14 (App Router) and TypeScript, styled with Tailwind CSS and shadcn/ui components. The max-w-md viewport constraint ensures a native app feel on mobile devices. The PWA is installable on Android and iOS. It is hosted on AWS and accessible via QR code for demonstration.

2.4.2 Backend and Database (Supabase / PostgreSQL)

The backend is implemented as Next.js API Routes and Server Actions, using Supabase (PostgreSQL) for persistent storage. Row Level Security (RLS) is enforced on all tables to ensure users can only access their own data. Key tables include: profiles, wallets, transactions (hash-chained ledger), fx_rates_cache, aml_flags, and polygon_anchors.

2.4.3 Authentication (Supabase Auth)

User authentication is handled by Supabase Auth using magic link email and phone OTP. No passwords are stored. Sessions are managed via Supabase's JWT-based session system. KYC status is stored in the user profile and gates access to transfer functionality.

2.4.4 FX Rate Engine

Live foreign exchange rates are fetched from the exchangerate.host API and cached hourly in the fx_rates_cache table to avoid rate limiting. A fallback to hardcoded rates is implemented for resilience. FX rates are displayed transparently to users on the FX Breakdown Screen before every transaction.

2.4.5 Blockchain Ledger and Polygon Anchoring

Every transaction in the platform is recorded in a SHA-256 hash-chained ledger stored in Supabase. Each row's hash is computed as $\text{SHA-256}(\text{row_data} + \text{prev_hash})$, making the chain tamper-evident. Every N transaction, the root hash is anchored to the Polygon Amoy testnet via the ethers.js library and a deployed smart contract. This provides public verifiability via PolygonScan while maintaining fast, low-cost user experience.

2.4.6 AI Insights Engine (Anthropic Claude API)

Personalized financial insight cards are generated using the Anthropic Claude API. Prompts are constructed with the user's transaction history, balance, and preferred language. Responses are streamed to the UI for a responsive feel. Insights include savings recommendations, remittance pattern analysis, and micro-investment suggestions.

2.5 Design and Implementation Constraints

All communication between the PWA client and the Supabase backend must be over HTTPS with TLS 1.2 or higher. Authentication tokens must be stored securely and not exposed to client-side JavaScript beyond what is required by the Supabase SDK. Because users may be controlling financial transactions remotely, special care in security must be implemented to prevent unauthorized access, account takeover, and fraudulent transfers. Failure to do so could result in financial loss to users and serious regulatory and reputational liability for SwiftX.

The platform must comply with RBI Money Transfer Service Scheme (MTSS) requirements for inward remittances to India, FATF AML/KYC recommendations, GDPR (for EU users), and India's Personal Data Protection Bill. The system must not store raw payment card data (PCI-DSS compliance is delegated to payment processors such as Stripe or Adyen for top-up flows).

2.6 Business Requirements

2.6.1 Business Opportunity

An estimated 250 million migrant workers worldwide send over \$700 billion in remittances annually, paying an average of 6.2% in fees representing over \$43 billion in annual fee burden on the world's most economically vulnerable workers. SwiftX addresses this market directly with a zero-fee, transparent-FX model that is competitive with Wise while being purpose-built for the migrant worker persona and India's regulatory framework.

2.6.2 Business Objectives and Success Criteria

a. Achieve a demonstrable end-to-end remittance flow completing in under 30 seconds on any mobile device, with live FX rates, zero fees, and a verifiable blockchain receipt.

b. Demonstrate revenue model viability: 0.5-1% transparent FX spread generates \$2.50 per average \$500 remittance; at 10 million transactions/month, this yields \$25 million monthly revenue without any user-facing fees.

2.6.3 Customer or Market Needs

a. Migrant workers currently rely on high-fee bank wires, hawala networks, or consumer remittance services with opaque pricing. There is an unmet need for a transparent, fee-free, mobile-first solution.

b. Many recipients are unbanked or underbanked. A mobile wallet receivable via UPI/IMPS eliminates the need for a traditional bank account for the recipient.

2.6.4 Business Risks

a. Regulatory risk: RBI MTSS compliance requires an Authorized Dealer Category II bank partnership. Failure to secure this partnership blocks INR payout in India.

b. Competition: Wise, PhonePe, and Paytm are established players. SwiftX's differentiation is the migrant-worker persona focus, AI guidance, and on-chain transparency.

c. FX volatility: Significant INR/USD rate movements could affect liquidity pool rebalancing costs. Mitigation: transparent spread disclosure and dynamic pool rebalancing.

2.7 User Documentation

Each SwiftX user will be onboarded through an in-app interactive tutorial covering wallet setup, KYC verification, sending money, and viewing transaction history. The corporate website will provide short video tutorials, a FAQ, and a list of supported currencies and payout methods. Comprehensive API and architecture documentation will be maintained for developers and compliance auditors.

2.8 Assumptions and Dependencies

The SwiftX PWA can be updated over-the-air via Next.js deployments on AWS without requiring users to take any action. The system assumes reliable HTTPS connectivity for the core transfer flow, with graceful degradation (cached FX rates, offline balance display) when connectivity is intermittent. The blockchain anchoring assumes access to the Polygon Amoy network; if the network is unreachable, the hash-chained ledger continues operating and anchoring retries when connectivity is restored.

3. System Features

3.1 Zero-Fee Remittance

Description

SwiftX enables users to send money across currencies with zero per-transaction fees. Money moves wallet-to-wallet within the SwiftX ledger, converting at a transparent FX rate with a disclosed 0.5-1% spread. Users see a clear FX breakdown before confirming every transfer.

Use Cases

Use Case ID	UC-1
Use Case Name	Send Cross-Border Remittance
Priority	High
Created By	SwiftX Development Team
Date Created	May 1, 2026
Goal	To allow a migrant worker to send money to a recipient in another country with zero fees, transparent FX rates, and instant settlement within the SwiftX ledger.
Actors	Primary: Sender (Migrant Worker) Secondary: Recipient (Family Member), FX Rate Engine, Hash-Chain Ledger
Assumptions	1. The sender has a verified KYC status (kyc_status = 'verified'). 2. The sender has a funded wallet in the source currency. 3. The recipient has a SwiftX wallet in the destination currency.
Pre-conditions	1. Sender is authenticated and KYC-verified. 2. Sender's wallet balance is sufficient for the transfer amount.
Primary Scenario	1. Sender taps 'Send Home' on the dashboard. 2. Sender enters the amount (e.g., INR 15,000) and selects the recipient. 3. The system displays the FX Breakdown Screen: source amount in USD, live exchange rate, disclosed FX margin, zero fee, INR amount to be received, estimated arrival time. 4. Sender confirms with biometric mock / PIN. 5. The system debits the sender's USD wallet and credits the recipient's INR wallet at the live FX rate. 6. A hash-chained transaction record is created. The SHA-256 hash is displayed as a receipt. 7. Every N transaction, the chain root hash is anchored to Polygon Amoy. The transaction ID links to PolygonScan. 8. Recipients see their INR balance update in real time.

	9. An AI insight card is generated for the sender.
Exceptions	3a. If the sender's wallet has insufficient balance, the system displays an error and prompts a top-up. 4a. If the FX API is unreachable, the system uses the most recently cached rate (up to 1 hour old) and displays a notice. 6a. If hash chain generation fails, the transaction is rolled back and the sender is notified.
Post-conditions	1. Sender's USD wallet is debited; recipient's INR wallet is credited. 2. Transaction is recorded in the immutable hash-chained ledger. 3. Both sender and recipient receive real-time balance and transaction notifications.
Frequency of Use	Several times per month per active sender.
Business Rules	B-1, B-2
Special Requirements	FX breakdown must display the exact exchange rate, the margin percentage, the fee (zero), and the amount the recipient will receive, before the sender confirms.
Notes and Issues	Polygon anchoring may have up to 5-transaction lag; this is by design and must be communicated in the receipt UI.

Use Case ID	UC-2
Use Case Name	Top Up Wallet
Priority	High
Created By	SwiftX Development Team
Goal	To allow a user to add funds to their SwiftX wallet from an external payment rail (bank transfer, card, or agent cash-in).
Actors	Primary: Sender Secondary: External Payment Processor (Stripe/Adyen for demo)
Pre-conditions	The user is authenticated and KYC-verified.
Primary Scenario	1. The user taps 'Top Up' on the dashboard. 2. The user selects the currency and top-up method. 3. The user enters the amount. 4. Payment processor (mocked in demo) processes the incoming payment. 5. The user's wallet balance is updated. Transaction recorded as tx_type = 'topup'.
Exceptions	4a. Payment processor returns an error. The wallet is not credited. Users are notified.

Post-conditions	The user's wallet balance reflects the topped-up amount. Transaction recorded in hash-chained ledger.
Frequency of Use	Monthly or before each remittance cycle.
Notes and Issues	In demo: wallets are pre-seeded. Production will integrate UPI (India) and Stripe/Adyen (international).

Use Case ID	UC-3
Use Case Name	Withdraw Funds to Bank / UPI
Priority	High
Created By	SwiftX Development Team
Goal	To allow a recipient to withdraw their SwiftX wallet balance to a linked bank account or UPI ID.
Actors	Primary: Recipient Secondary: UPI/IMPS Rail (mocked for demo)
Pre-conditions	The recipient is authenticated, KYC-verified, and has a positive wallet balance.
Primary Scenario	1. Recipient taps 'Withdraw'. 2. Recipient selects amount and linked UPI ID / bank account. 3. The system initiates withdrawal. Transaction recorded as tx_type = 'withdraw'. 4. The recipient's wallet balance is debited. UPI/IMPS rail transfer is initiated (mocked in demo).
Exceptions	2a. If UPI/IMPS rail is unreachable, withdrawal is queued and retried. Users are notified of pending status.
Post-conditions	Wallet balance reduced. Transaction immutably recorded. Funds delivered to an external account.
Frequency of Use	As needed by the recipient.

3.1.1 FX Breakdown Display

3.1.1.1 Before every remittance confirmation, the system shall display a FX Breakdown Screen showing: the amount in the sender's currency, the live exchange rate (fetched within the last hour), the disclosed FX margin percentage (0.5–1%), the fee amount (always \$0.00), the amount the recipient will receive in the destination currency, and the estimated arrival time.

3.1.1.2 If the FX rate used is from the cache (not live), the system shall display a notice indicating the rate age (e.g., 'Rate as of 15 minutes ago').

3.1.1.3 The FX Breakdown Screen shall calculate and display the savings compared to the average bank rate for the same corridor.

3.1.2 Hash-Chained Transaction Receipt

3.1.2.1 Every completed transaction shall be assigned a SHA-256 hash computed as SHA-256(transaction_data + previous_transaction_hash), linking it immutably to the prior transaction.

3.1.2.2 The transaction receipt screen shall display the transaction hash, timestamp, sender wallet ID, recipient wallet ID, amount sent, amount received, FX rate applied, and Polygon anchor transaction hash (when available).

3.1.2.3 Every N transactions (configurable, default = 5), the current chain root hash shall be submitted to the Polygon Amoy smart contract. The resulting Polygon transaction hash shall be stored in the polygon_anchors table and displayed as a clickable PolygonScan link on the receipt.

3.1.3 Multi-Currency Wallet

3.1.3.1 Each user may hold wallet balances in multiple currencies simultaneously (USD, AED, INR, and additional currencies as configured).

3.1.3.2 Wallet balances shall be updated atomically within a database transaction: debit of source wallet and credit of destination wallet shall succeed or fail together, with no partial state.

3.1.3.3 The dashboard shall display all of a user's wallet balances in real time using Supabase realtime subscriptions.

3.2 User Authentication and KYC Verification

Description

SwiftX uses passwordless authentication (magic link and phone OTP) to reduce friction for low-literacy users. Before a user can initiate transfers, they must complete KYC verification by submitting a government-issued ID document. This is required by RBI MTSS guidelines and FATF AML/KYC recommendations.

Use Cases

Use Case ID	UC-4
Use Case Name	Register and KYC Verify
Priority	High
Goal	To create a new SwiftX account and complete KYC verification so the user is eligible to send or receive money.

Actors	Primary: New User Secondary: KYC Verification Service
Pre-conditions	The user has a valid mobile phone number or email address.
Primary Scenario	1. The user opens the SwiftX PWA and taps 'Get Started'. 2. The user enters their full name, phone number, country of residence, and preferred language. 3. The user receives an OTP via SMS and enters it to authenticate. 4. A new user profile is created with kyc_status = 'pending'. 5. Users are prompted to upload a government-issued ID document (passport, national ID, or driver's license). 6. Document is stored securely. KYC status is updated to 'verified' upon approval (or 'rejected' if invalid). 7. The user receives an in-app notification of KYC status.
Exceptions	3a. OTP is entered incorrectly three times. The user is locked out for 10 minutes. 6a. The KYC document is unreadable or mismatched. Users are prompted to resubmit.
Post-conditions	The user account exists with verified KYC status. Users may initiate transfers up to the regulatory limit.
Business Rules	B-3
Special Requirements	KYC documents must be encrypted at rest and in transit. Raw document data is never exposed via API. Access restricted to compliance officers only.

Use Case ID	UC-5
Use Case Name	Log Into SwiftX
Priority	High
Goal	To authenticate an existing user and access their SwiftX account.
Actors	Primary: Registered User
Pre-conditions	The user has a registered account.
Primary Scenario	1. The user opens PWA and taps 'Log In'. 2. The user enters a phone number or email. 3. The system sends OTP. 4. The user enters OTP. 5. System validates credentials. Session is created. Dashboard is displayed.
Exceptions	4a. OTP/magic link expired (>10 minutes). The user requests a new one. 4b. OTP entered incorrectly three times in a row. Account temporarily locked. Users must contact support.

Post-conditions	The user is authenticated. Session token is valid for 24 hours or until sign-out. Dashboard shows current wallet balances.
Special Requirements	Session tokens must be stored in httpOnly cookies, not localStorage, to mitigate XSS risk. Concurrent sessions from multiple devices are allowed.

3.2.1 Authentication Requirements

3.2.1.1 The system shall support passwordless authentication via SMS OTP and email magic link.

3.2.1.2 OTPs shall be exactly 6 digits, expire after 10 minutes, and be invalidated after a single successful use.

3.2.1.3 After three consecutive failed OTP attempts, the user's account shall be temporarily locked for 10 minutes, and a notification shall be sent.

3.2.1.4 Sessions shall expire after 24 hours of inactivity, requiring re-authentication.

3.2.2 KYC Verification Requirements

3.2.2.1 All users must complete KYC verification before initiating any transfer or withdrawal. Users with kyc_status = 'pending' or 'rejected' shall see a clear prompt to complete or resubmit KYC.

3.2.2.2 The system shall accept government-issued photo ID documents (passport, national ID, driver's license) as JPEG, PNG, or PDF uploads. For live demo, a webcam capture option via getUserMedia shall also be available.

3.2.2.3 KYC document storage shall be encrypted at rest using AES-256. Document URLs shall not be publicly accessible; access requires a signed URL valid for 15 minutes.

3.2.2.4 The system shall store and audit all KYC status changes with timestamps and the identity of the reviewing compliance officer.

3.3 AI-Powered Financial Insights

Description

SwiftX uses the Anthropic Claude API to generate personalized financial insight cards for each user. Insights are triggered after transactions, on dashboard load, or on user request. They are delivered in the user's preferred language and include savings analysis, remittance pattern insights, and actionable financial guidance.

Use Case ID	UC-6
Use Case Name	View AI Financial Insight

Priority	Medium
Goal	To provide the user with personalized, actionable financial guidance based on their transaction history and balance.
Actors	Primary: User Secondary: Claude AI (Anthropic API)
Pre-conditions	The user is authenticated. At least one completed transaction exists.
Primary Scenario	1. The user completes a remittance transaction. 2. The system constructs a prompt including the user's transaction history (last 90 days), total amount sent, destination country, and preferred language. 3. Claude API is called with the prompt. Response is streamed to the UI. 4. An insight card appears, e.g.: 'Ravi, you have sent \$1,200 home this quarter. Saving \$20 per week could build a \$1,040 emergency fund by year-end.' 5. Users can tap the card to initiate a recommended action (e.g., start SIP).
Exceptions	3a. Claude API is unavailable. A graceful fallback message is shown: 'Insights temporarily unavailable. Check back soon.'
Post-conditions	Users have received actionable financial guidance in their language.
Special Requirements	AI prompts must never include raw KYC document content. Transaction data sent to the AI must be anonymized of recipient's full financial details.

3.3.1 AI Insights Functional Requirements

3.3.1.1 The system shall generate personalized financial insight cards using the Anthropic Claude API after every completed transaction and on dashboard load (if last insight is older than 24 hours).

3.3.1.2 Insights shall be generated in the user's preferred language as stored in the profiles table (English or Hindi in v1.0).

3.3.1.3 Insight generation shall use streaming (server-sent events) so the card text appears progressively, providing responsive UX on slow mobile connections.

3.3.1.4 Each insight card shall include at least one actionable call-to-action button (e.g., 'Start SIP', 'View savings goal', 'Learn more').

3.4 AML Compliance and Admin Dashboard

Description

SwiftX includes an automated AML rules engine that screens every transaction against configurable risk rules (amount thresholds, velocity checks, high-risk corridors). Flagged transactions are surfaced to compliance officers via the Admin Dashboard. The audit log provides a complete, immutable record of all transactions and compliance actions.

Use Case ID	UC-7
Use Case Name	AML Flag Review (Admin)
Priority	High
Goal	To allow a compliance officer to review AML-flagged transactions and take appropriate action.
Actors	Primary: Compliance Officer
Pre-conditions	The compliance officer is authenticated with the admin role. At least one AML flag exists in the aml_flags table.
Primary Scenario	1. The compliance officer logs into the Admin Dashboard. 2. The system displays all unresolved AML flags sorted by severity (high, medium, low) and timestamp. 3. Officer reviews the flagged transaction details: amount, sender, recipient, rule triggered, transaction hash. 4. The officer marks the flag as resolved with notes, or escalates to law enforcement as required.
Exceptions	4a. Escalation to law enforcement triggers an automated Suspicious Activity Report (SAR) filing workflow.
Post-conditions	The AML flag is marked resolved with an audit trail. Transaction remains immutably recorded in a hash-chained ledger.
Special Requirements	All compliance officer actions (flag resolution, escalation, notes) must be logged with timestamp, officer ID, and action taken, and must be immutable.

3.4.1 AML Rules Engine Requirements

3.4.1.1 The system shall automatically flag any single transaction exceeding USD 1,000 (or equivalent) with severity 'medium', and any transaction exceeding USD 3,000 with severity 'high'.

3.4.1.2 The system shall flag velocity violations: more than 5 transactions from a single user within a 24-hour period shall trigger a 'velocity_check' AML flag.

3.4.1.3 All AML flags shall be stored in the aml_flags table with transaction_id, rule_triggered, severity, resolved status, and notes.

3.4.1.4 The Admin Dashboard shall provide real-time visibility into liquidity pool balances by currency, total transaction volume, and unresolved AML flags.

3.5 Micro-Investment Products

Description

SwiftX provides accessible micro-investment features for underbanked users, including Systematic Investment Plans (SIPs) starting at INR 100/week. These features allow users to build long-term savings habits directly from their remittance app, without needing a separate investment account.

Use Case ID	UC-8
Use Case Name	Start Micro-Investment SIP
Priority	Medium
Goal	To allow a user to initiate a recurring micro-investment plan from their SwiftX wallet balance.
Actors	Primary: User (Recipient in Home Country)
Pre-conditions	The user is authenticated, KYC-verified, and has a positive INR wallet balance.
Primary Scenario	1. The user taps 'Start SIP' from an AI insight card or investment menu. 2. The user selects the investment amount (minimum INR 100), frequency (weekly/monthly), and investment basket. 3. User confirms. The system creates a recurring investment schedule. 4. On each scheduled date, the system transfers the investment amount from the user's wallet to the investment basket, recording the transaction as tx_type = 'investment'.
Exceptions	4a. Wallet balance is insufficient on the scheduled date. Investment is skipped for that cycle and the user is notified.
Post-conditions	Recurring investment schedule is active. The user's investment portfolio is updated after each cycle.
Special Requirements	Minimum investment amount is INR 100. Users must be able to pause or cancel their SIP at any time.

3.5.1 Micro-Investment Functional Requirements

3.5.1.1 The system shall support recurring investment plans (SIPs) with a minimum investment of INR 100, with weekly or monthly frequency.

3.5.1.2 Investment transactions shall be recorded in the transactions table with tx_type = 'investment' and included in the hash-chained ledger.

3.5.1.3 Users shall be able to view their investment portfolio balance, total invested, and estimated returns on the dashboard.

3.5.1.4 Users shall be able to pause, modify, or cancel their SIP at any time with immediate effect.

3.6 Multilingual Interface and Voice Input

Description

SwiftX supports English and Hindi in version 1.0, with the architecture designed for easy extension to additional languages. Voice input via the Web Speech API allows low-literacy users to navigate the app by voice, improving accessibility for underserved populations.

3.6.1 Multilingual and Voice Functional Requirements

3.6.1.1 The system shall support English (en) and Hindi (hi) as selectable interface languages in version 1.0. The user's preferred language shall be stored in the profiles table.

3.6.1.2 Language selection shall be accessible from the main navigation with a single tap and shall take effect immediately without requiring a page reload.

3.6.1.3 AI insight cards shall be generated in the user's preferred language by passing the language preference to the Claude API prompt.

3.6.1.4 Voice command input shall be available on supported browsers via the Web Speech API, allowing users to navigate to key screens (e.g., 'Send money', 'Check balance', 'View history') by voice.

3.6.1.5 If voice input is unavailable (browser does not support Web Speech API), the voice input button shall be hidden without error.

4. Non-Functional Requirements

4.1 Process Requirements

4.1.1 Management Requirements

4.1.1.1 The SRS document shall be completed and reviewed before development begins.

4.1.1.2 All code shall be version-controlled using Git and hosted on a private GitHub repository with branch protection rules on main.

4.1.1.3 A working end-to-end demo must be available by Hour 12 of the hackathon schedule, tested on a real mobile device.

4.1.1.4 Production deployment to AWS must be completed by Hour 20 of the hackathon schedule.

4.1.1.5 All environment variables (API keys, database credentials) must be stored in AWS environment configuration, never committed to the repository.

4.1.2 Implementation Requirements

4.1.2.1 The frontend shall be developed using Next.js 14 (App Router) with TypeScript.

4.1.2.2 The UI shall be built with Tailwind CSS and shadcn/ui components for consistent, production-quality styling.

4.1.2.3 The backend shall use Supabase (PostgreSQL) with Row Level Security enforced on all user-data tables.

4.1.2.4 Blockchain anchoring shall use ethers.js to interact with the Polygon Amoy network.

4.1.2.5 AI insights shall be generated using the Anthropic Claude API (claude-sonnet-4-20250514 model).

4.1.3 Standards Requirements

4.1.3.1 The development process shall follow an agile sprint model with daily standups and a shared task board.

4.1.3.2 Code shall follow TypeScript strict mode and ESLint rules. No type: any shall be used in production code paths.

4.1.3.3 All database schema changes shall be tracked as Supabase migrations committed to the repository.

4.2 Product Requirements

4.2.1 Usability Requirements

4.2.1.1 A new user with no prior experience with the app shall be able to complete their first remittance within 3 minutes of onboarding, as measured by usability testing.

4.2.1.2 The app shall be fully usable on a smartphone screen (minimum 360px width) without horizontal scrolling.

4.2.1.3 All primary actions (Send, Top Up, Withdraw, View History) shall be reachable within 2 taps from the main dashboard.

4.2.1.4 The FX Breakdown Screen shall display the zero-fee and live exchange rate information clearly enough that a user can explain the benefit to a family member without reading instructions.

4.2.2 Performance Requirements

Static performance:

4.2.2.1.1 The Next.js PWA bundle shall have a first contentful paint (FCP) of under 2 seconds on a 3G mobile connection.

4.2.2.1.2 The Supabase database shall require no more than 2GB storage for the first 1 million transactions including audit logs.

Dynamic performance:

4.2.2.2.1 The complete remittance flow (from 'Send' tap to confirmed receipt display) shall complete in under 30 seconds under normal conditions.

4.2.2.2.2 Real-time wallet balance updates via Supabase realtime shall propagate to the recipient's screen within 2 seconds of transaction confirmation.

4.2.2.2.3 FX rate fetch and display on the Breakdown Screen shall occur within 1 second of the user entering the transfer amount.

4.2.2.2.4 AI insight card generation shall begin streaming within 2 seconds of the request and complete within 10 seconds.

4.2.2.2.5 Polygon Amoy anchoring shall complete within 60 seconds of the anchor transaction being submitted; failures shall be retried up to 3 times before logging an error.

4.2.3 Reliability Requirements

4.2.3.1 There must be no possibility of a partial transfer state: a transfer that debits the sender without crediting the recipient. All wallet debit and credit operations must be wrapped in an atomic database transaction (Supabase RPC with BEGIN / COMMIT / ROLLBACK).

4.2.3.2 The hash chain integrity must be verifiable at any time: a chain verification routine shall confirm that every transaction's hash correctly links to the prior transaction, and any break in the chain must trigger an immediate alert to the compliance officer.

4.2.3.3 All possible errors and exceptions in critical paths (transfer, KYC, AML flagging) must be caught, logged to the error monitoring system, and reported to the development team without causing a user-facing system crash.

4.2.4 Availability

4.2.4.1 The system shall target 99.9% uptime (less than 9 hours downtime per year) after production launch, enabled by AWS auto-scaling and Supabase managed infrastructure.

4.2.4.2 Planned maintenance windows shall not exceed 30 minutes and must be communicated to users at least 24 hours in advance.

4.2.5 Platform Constraints

4.2.5.1 The PWA shall function on Chrome (Android), Safari (iOS 15+), and Firefox (Android) without native app installation.

4.2.5.2 The system shall require Node.js 18+ on the server side. Supabase client SDK version 2.x shall be used.

4.2.5.3 The system shall not rely on any deprecated browser APIs. Progressive enhancement shall ensure core functionality (sending money, viewing balance) works without JavaScript-only features such as voice input.

4.2.6 Modifiability

4.2.6.1 Adding support for a new currency (e.g., GBP, EUR) shall require no more than 1 person-day of engineering effort: adding the currency to the wallets schema, FX rate cache configuration, and UI selectors.

4.2.6.2 Adding a new supported language shall require no more than 2 person-days: adding translation strings to the i18n configuration and testing AI prompt output in the new language.

4.3 External Requirements

4.3.1 Business Rules

ID	Rule Definition	Type	Static or Dynamic	Source
B-1	Every transaction must be recorded in the hash-chained ledger before the user sees a success confirmation.	Constraint	Static	Product Policy
B-2	FX margin must be disclosed to the user on the FX Breakdown Screen before every transfer. The margin shall not exceed 1% of the transfer amount.	Constraint	Static	Corporate Policy
B-3	No user may initiate a transfer until KYC status is 'verified'. Users with 'pending' or 'rejected' KYC are restricted to wallet top-up only.	Constraint	Static	RBI MTSS
B-4	There is no per-transaction fee charged to the user. Revenue is generated solely via FX spread, float interest, and premium features.	Policy	Static	Business Model

4.3.2 Legal Constraints

4.3.2.1 All inward remittances to India must be processed in compliance with the Reserve Bank of India Money Transfer Service Scheme (RBI MTSS) through an authorized AD-II banking partner.

4.3.2.2 All users must complete KYC verification in accordance with FATF recommendations before initiating any transfer.

4.3.2.3 Under privacy laws (GDPR and India PDPB), user personal data and KYC documents must be stored only within approved jurisdictions, encrypted at rest and in transit, and never sold or shared with third parties without explicit user consent or a court order.

4.3.2.4 Transactions exceeding regulatory reporting thresholds must be automatically flagged and reported to the competent authority in compliance with AML regulations.

4.3.2.5 Compliance officers may access user transaction data only for AML/KYC review purposes. Access to surveillance data outside of flagged compliance events is prohibited.

4.3.3 Economic Constraints

4.3.3.1 The development budget for version 1.0 of SwiftX shall be managed to fit within the hackathon's resource constraints, utilizing free-tier and open-source services wherever possible (Supabase free tier, exchangerate.host free tier, Polygon Amoy testnet).

4.3.4 Interoperability Requirements

4.3.4.1 User Interfaces

4.3.4.1.1 The primary user interface shall be the SwiftX Progressive Web App, accessible on any modern mobile browser. The UI must support touch gestures, large tap targets (minimum 44x44px), and high-contrast text for accessibility.

4.3.4.1.2 The Admin Dashboard shall be accessible to compliance officers via a separate authenticated route (/admin) and shall display liquidity pool status, AML flags, and the full transaction audit log.

4.3.4.1.3 The AI insight card interface shall support streaming text rendering with a typing-indicator animation while the Claude API response is being generated.

5. Validation Criteria

5.1 Final testing and acceptance of the SwiftX platform shall be evaluated against the success criteria defined in Section 16 of the Implementation Plan and the requirements stated in this SRS.

5.2 Criteria to address include system inputs, system processes, and system outputs.

5.3 Testing and acceptance shall follow standards appropriate for a financial transaction platform operating in the public fintech space.

5.4 Testing and acceptance processes shall be expressed by:

- a. Observing end-to-end transfer flows on real mobile devices (not emulators).
- b. Comparing known inputs (seeded user balances, FX rates) and expected outputs (credited balances, hash receipts) with actual system outputs.
- c. Getting required results regarding: (i) Transfer accuracy, (ii) Transfer speed (under 30 seconds), (iii) Hash chain integrity, (iv) Polygon anchor public verifiability, (v) AML flag correctness, (vi) AI insight language correctness, (vii) Compliance with all requirements stated in this SRS.

5.5 Must pass the following testing sequence:

- a. Functionality Testing:** Confirm all use cases (UC-1 through UC-8) work end-to-end as specified, including exception paths (insufficient balance, failed AML rule, KYC rejection).
- b. Security Testing:** Verify RLS policies block cross-user data access. Verify that unauthenticated API requests are rejected. Verify KYC documents are inaccessible without a signed URL.
- c. Performance Testing:** Demonstrate that the full transfer flow completes in under 30 seconds. Verify FX rate display latency is under 1 second. Verify realtime balance update propagates to the recipient within 2 seconds.
- d. Blockchain Integrity Testing:** Run the chain verification routine on all test transactions. Verify that tampering with any transaction record breaks the chain. Verify that the Polygon Amoy anchor transaction is publicly visible on PolygonScan.
- e. Compliance Testing:** Verify that a transfer attempt by a non-KYC user is blocked. Verify that a transaction above the AML threshold generates a flag in the `aml_flags` table. Verify the admin dashboard surfaces the flag correctly.
- f. Usability Testing:** A first-time user with no briefing shall be able to complete a remittance within 3 minutes. Language switching shall be verified in Hindi.

Appendix A: Architecture Diagrams

Figure A-1 Architecture Diagram

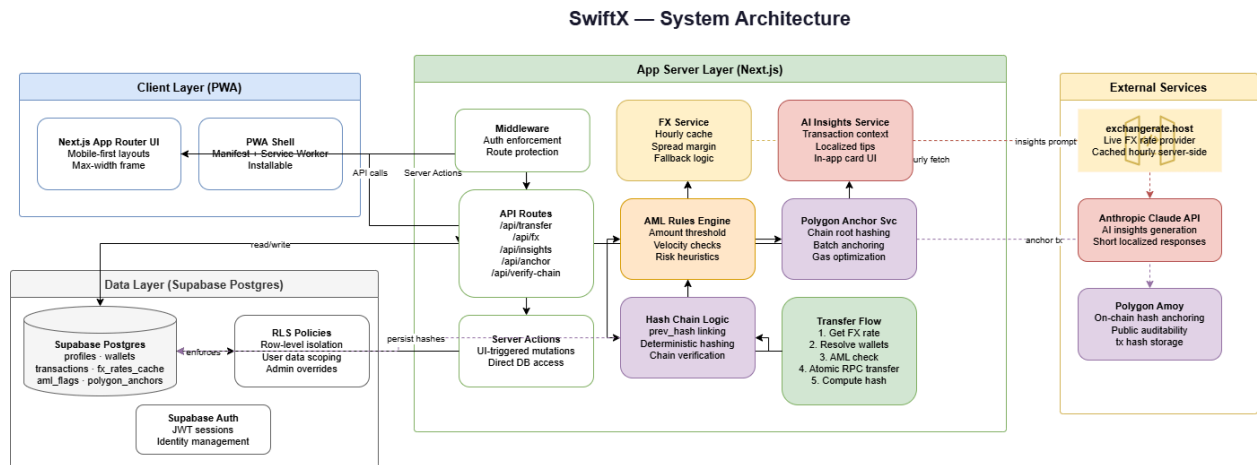


Figure A-2 - Activity Diagram

SwiftX – Send Remittance: Activity Diagram

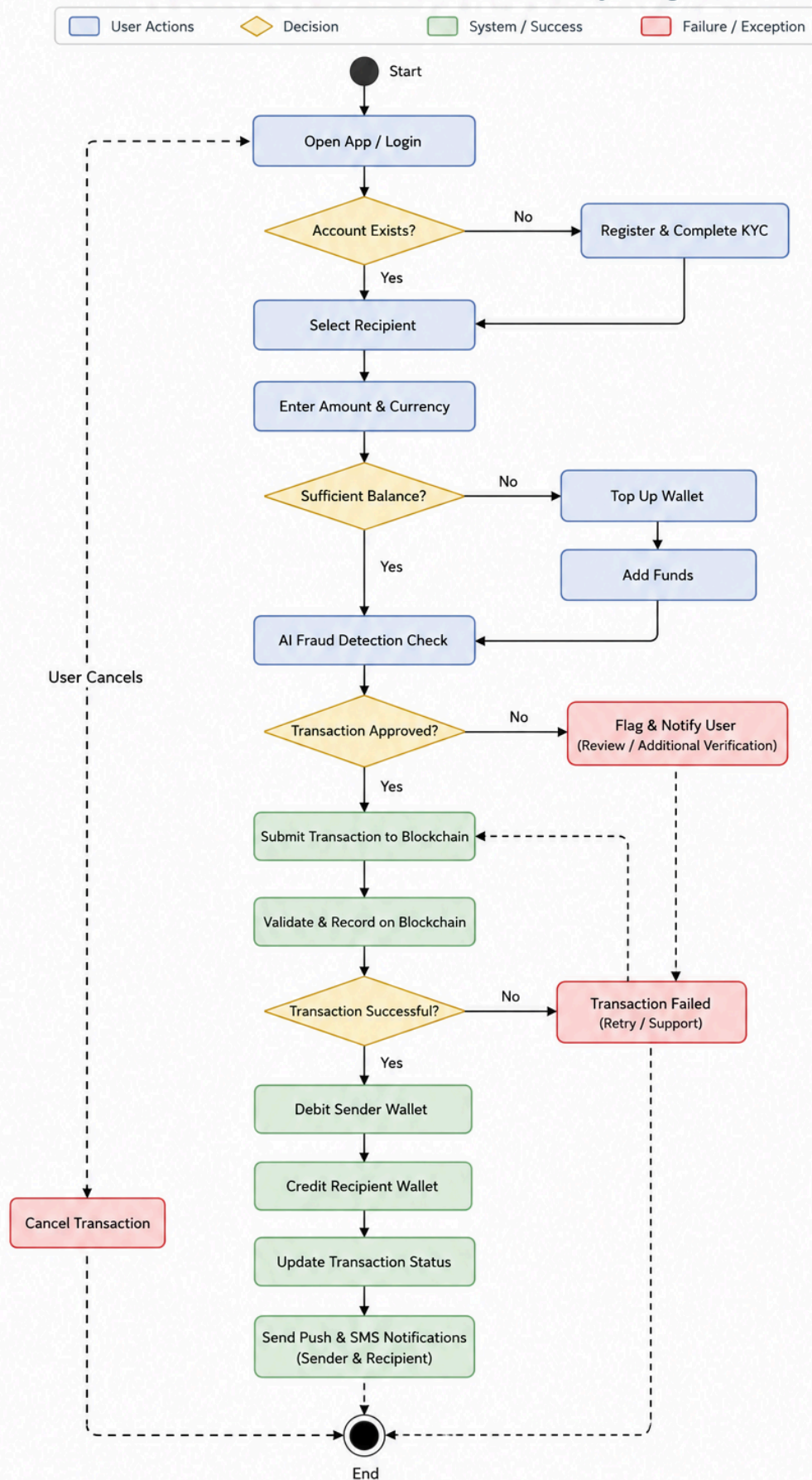


Figure A-3 – Component Diagram

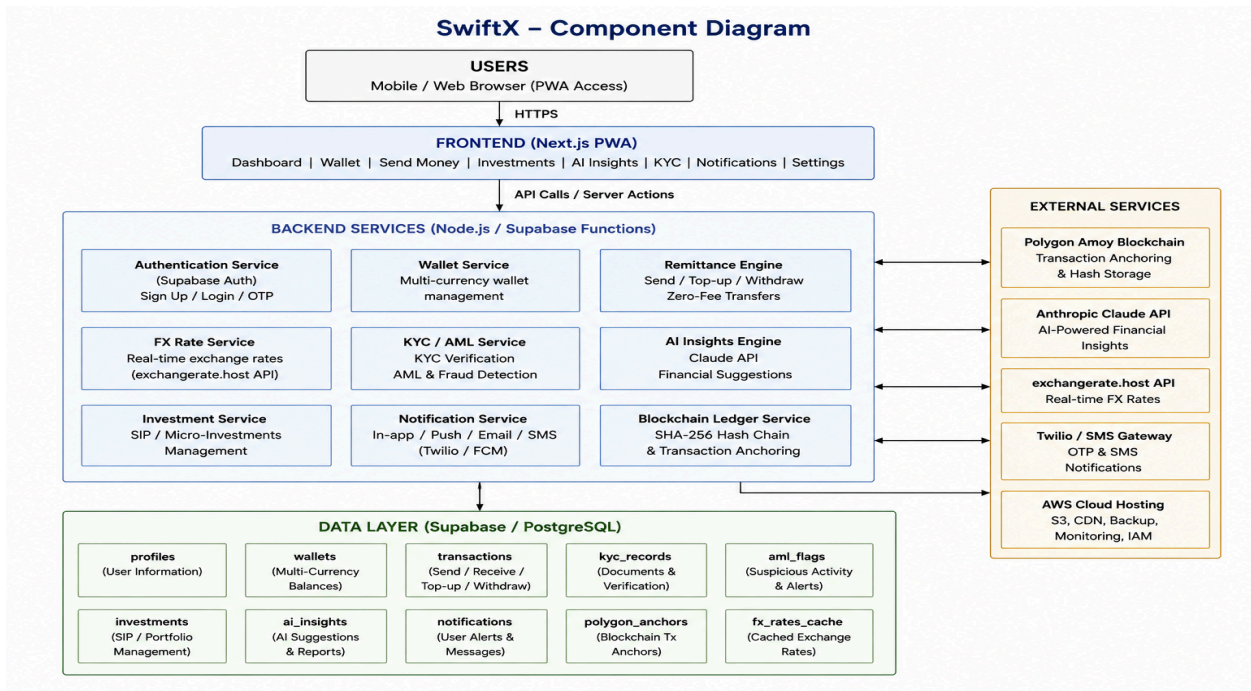


Figure A-4 - ER Diagram

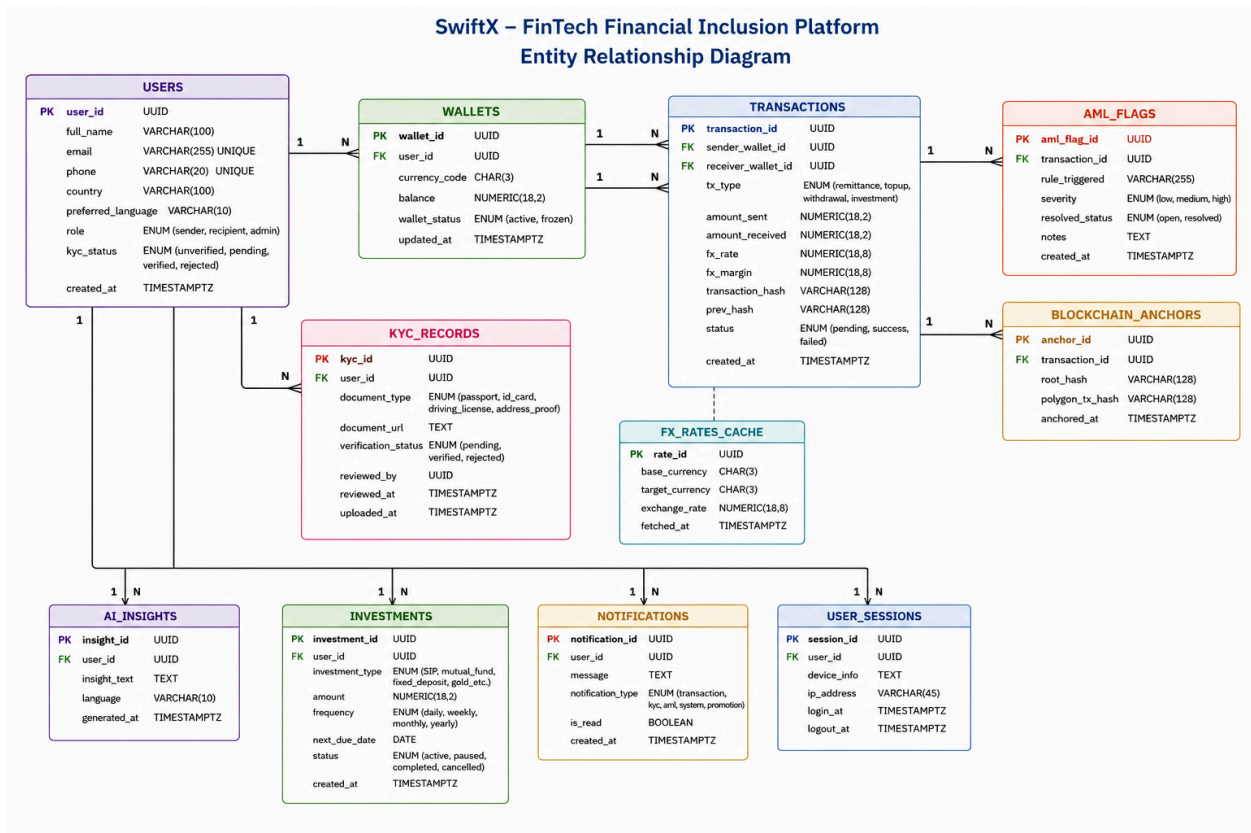


Figure A-5 – Sequence Diagram

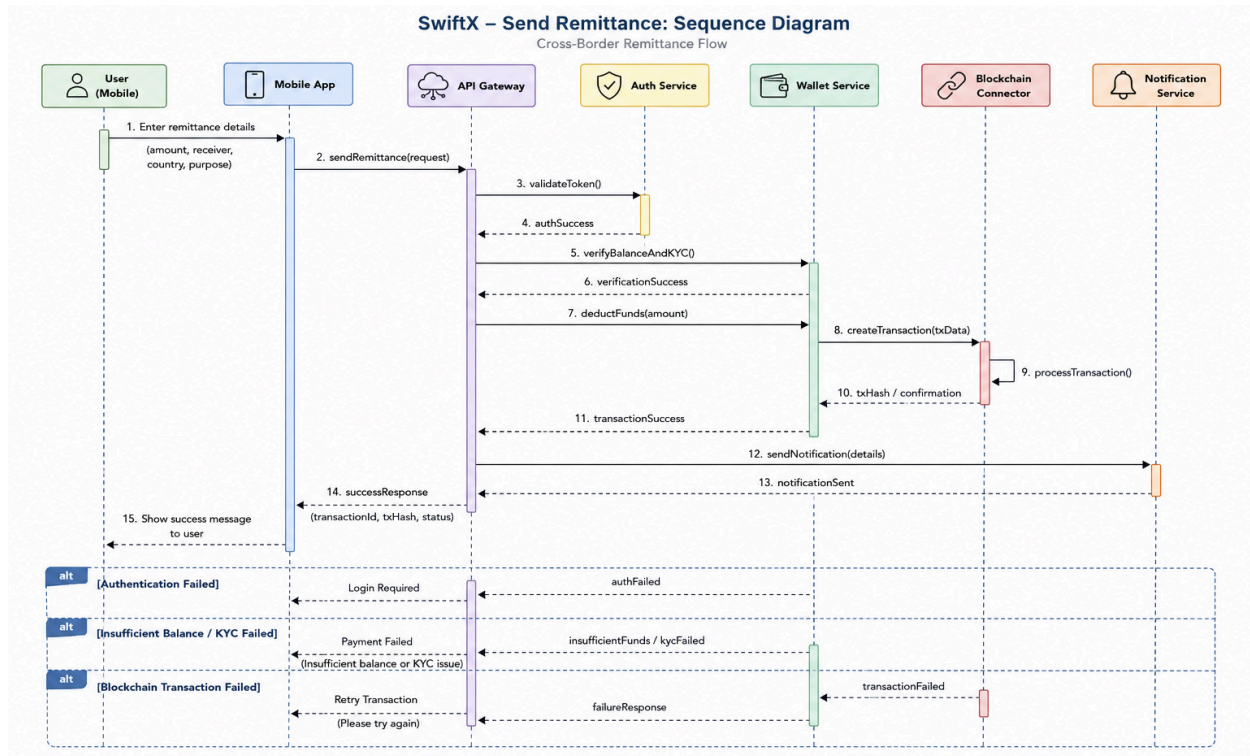


Figure A-6 – State Diagram

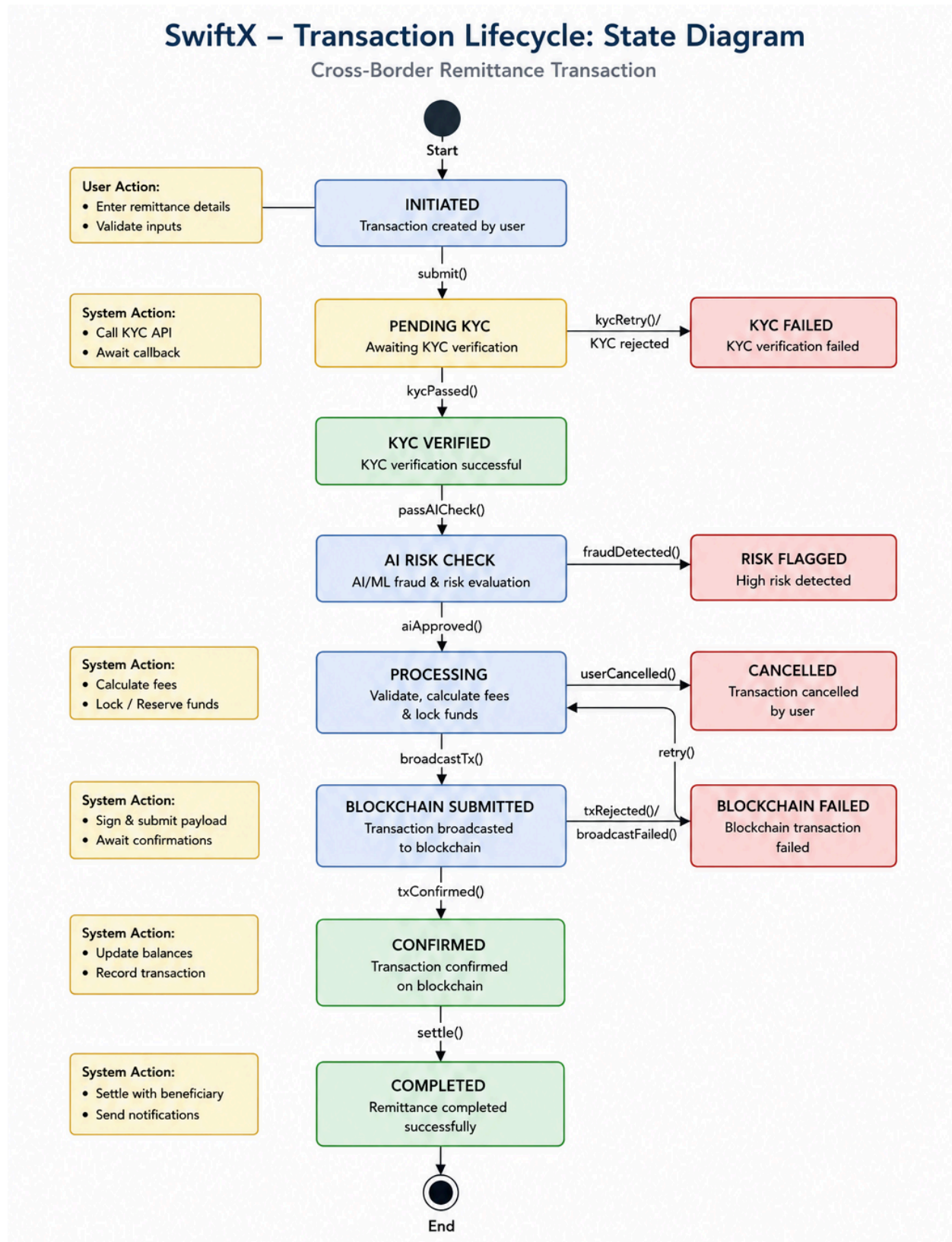


Figure A-7 – Use Case

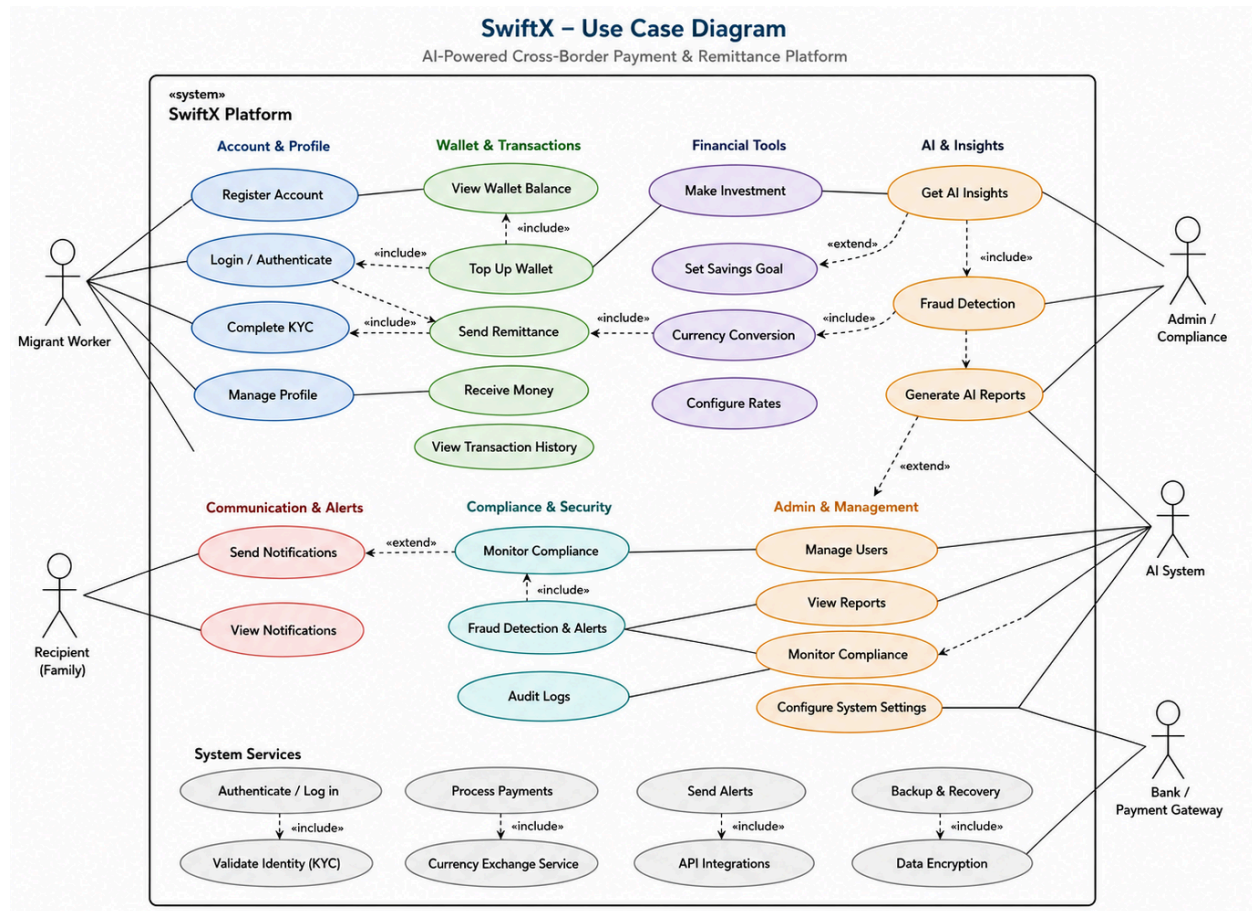
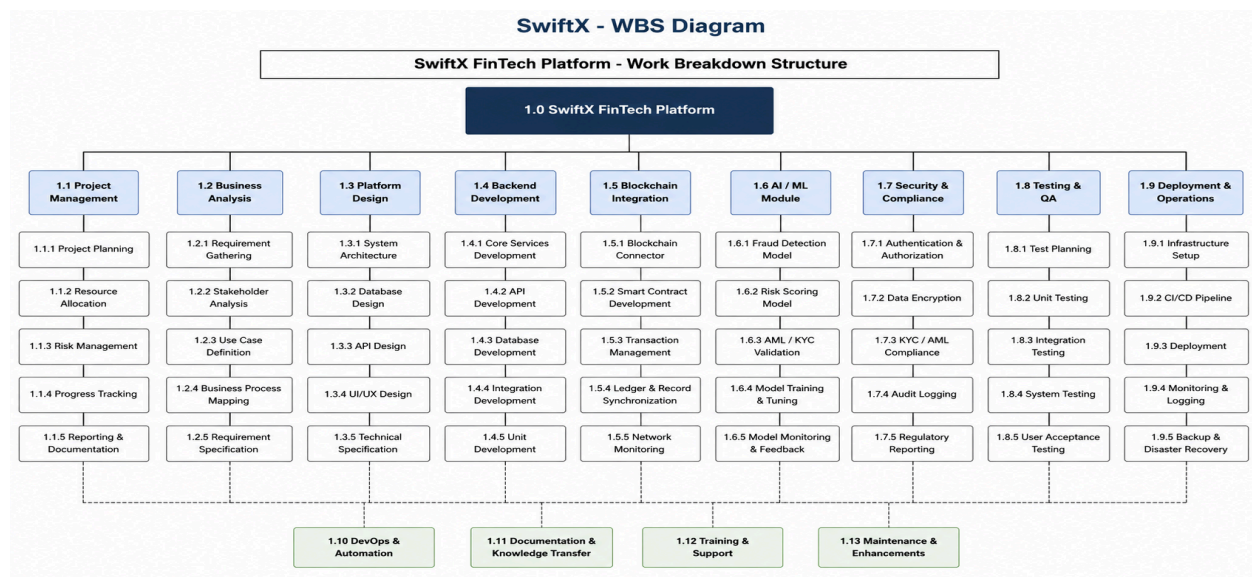


Figure A-8 – Work Breakdown Structure



Appendix B: Glossary and Acronyms

Term	Definition
AED	United Arab Emirates Dirham — currency used in Dubai, a key source country for SwiftX senders.
AML	Anti-Money Laundering — regulatory framework to detect and prevent financial crimes.
FX	Foreign Exchange — the conversion rate between two currencies (e.g., USD to INR).
Hash Chain	A cryptographic data structure where each record contains the SHA-256 hash of the prior record, making tampering detectable.
INR	Indian Rupee — the destination currency for remittances to India.
KYC	Know Your Customer — the process of verifying a user's identity using government-issued documents, required by AML regulations.
MTSS	Money Transfer Service Scheme — RBI framework governing inward cross-border remittances to India.
Polygon Amoy	A public EVM-compatible blockchain testnet used by SwiftX to publicly anchor the hash chain root, enabling external verifiability via PolygonScan.
PWA	Progressive Web Application — a web app that behaves like a native mobile app, installable without an app store.
RBI	Reserve Bank of India — India's central bank and financial regulator.
RLS	Row Level Security — a PostgreSQL feature enforced by Supabase to restrict data access so users can only read/write their own records.
SIP	Systematic Investment Plan — a recurring micro-investment of a fixed amount at regular intervals.
UPI	Unified Payments Interface — India's real-time payment rail for bank account transfers.
USD	United States Dollar — the primary source currency for migrant workers in the Gulf.

Acronym	Expansion
API	Application Programming Interface
AWS	Amazon Web Services
FATF	Financial Action Task Force
FCP	First Contentful Paint
GDPR	General Data Protection Regulation
JWT	JSON Web Token

OTP	One-Time Password
PDPB	Personal Data Protection Bill (India)
SAR	Suspicious Activity Report
SRS	Software Requirements Specification
TLS	Transport Layer Security